Compiler Design Project

Compiler Optimization Playground: An Interactive, Browser-Based Visualization of Three-Address Code Generation, Control Flow Analysis, and Optimization

1st Author Name
Raghav Gupta
(24BCE2510)
VIT,Vellore
Compiler Design

2nd Author Name
Kriday Sharma
(24BCE2351)
VIT,Vellore
Compiler Design

3rd Author Name
Kritagy Kaushik
(24BCI0012)
VIT,Vellore
Compiler Design

4th Author Name
Aryan Dabas
(24BAI0326)
VIT,Vellore
Compiler Design

***Abstract*—This paper presents the Compiler Optimization Playground, an interactive educational tool that executes a complete compiler back-end pipeline entirely inside a web browser. Users type their own C-like source programs into an editor; the system then tokenizes and parses the code, lowers it to Three-Address Code (TAC), partitions the TAC into basic blocks, builds a Control Flow Graph (CFG) and per-block expression Directed Acyclic Graphs (DAGs), and applies five modular optimization passes: constant folding and propagation, common subexpression elimination, dead code elimination, strength reduction, and loop-invariant code motion. Every optimized program is checked by a virtual machine that executes the baseline and optimized TAC and confirms identical console output and return values. A step-by-step inspector and side-by-side comparison view make the effect of each pass visible. The tool is implemented in TypeScript with React and is validated by an automated suite of 34 tests. On the benchmark program used for faculty review, the pipeline reduces eight initial TAC instructions to a single instruction that returns the verified value 50.**

***Keywords*—compiler design, code optimization, three-address code, control flow graph, directed acyclic graph, educational software visualization**

I. INTRODUCTION

Compiler construction is one of the most abstract subjects in a computer science curriculum. Students are shown intermediate representations, graphs, and data-flow equations on slides, but rarely get to watch a real program travel through each transformation. Production compilers expose their intermediate forms only through command-line flags and dense textual dumps, which are difficult for learners to interpret.

The Compiler Optimization Playground was built for the BCSE307P Compiler Design Laboratory to close this gap. Its single highest priority is that users can write *their own* custom C-like programs and see the compiler genuinely process them, rather than replaying canned examples.

A. Problem Statement

Learners need a way to observe, step by step, how source text becomes optimized low-level code, and to trust that each transformation is correct. Static diagrams cannot answer questions such as “what happens to my own loop?”, while industrial toolchains require installation, build configuration, and expertise in reading their output.

B. Objectives and Contributions

- a textbook-faithful, eight-phase compiler pipeline implemented in TypeScript that runs entirely client-side;
- five independently toggleable optimization passes with dynamic instruction-count metrics;
- a virtual machine semantic verifier that checks, by execution, that optimization preserves program behavior; and
- interactive CFG, DAG, and side-by-side comparison visualizations with a step-by-step pass inspector.

The rest of this paper is organized as follows. Section II gives background, Section III describes the architecture, Sections IV to VII detail each compiler phase, Section VIII discusses correctness, complexity, and design decisions, Section IX describes the interface, Section X covers implementation and testing, Section XI presents results, Section XII presents extended case studies, Section XIII covers diagnostics, Section XIV compares the tool with existing approaches, Section XV describes deployment, Section XVI describes educational use, Section XVII records lessons learned, and Sections XVIII and XIX give limitations and conclusions.

II. BACKGROUND AND RELATED WORK

A. Three-Address Code

Three-address code is a linear intermediate representation in which every instruction has at most one operator and at most three operands, for example `x = y + z`. Complex expressions are broken into sequences of such instructions using compiler-generated temporaries. The form is close to

machine code, easy to analyze, and independent of the source language [1], [2].

B. Basic Blocks and Control Flow

A basic block is a maximal sequence of instructions with a single entry at its first instruction and a single exit at its last. Blocks are linked by edges that represent possible transfers of control, forming a control flow graph [1]. The CFG is the basis for loop detection and for every global analysis.

C. DAGs and Value Numbering

Within a block, a directed acyclic graph merges identical computations into one node, exposing common subexpressions and dead values at a glance. Local value numbering is the equivalent table-based technique [1], [3].

D. Data-Flow Analysis

Liveness is a backward data-flow problem. A variable is live at a point if its current value may be read before it is overwritten. For a block B with successors S, the standard equations are

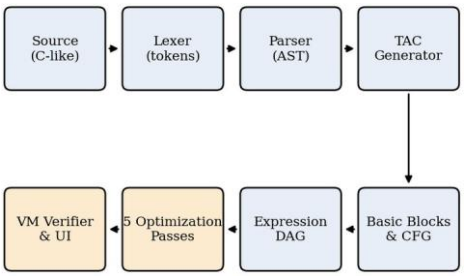


Fig. 1. Compilation pipeline of the Compiler Optimization Playground.

$$LIVE_in(B) = use(B) \cup (LIVE_out(B) - def(B)) \quad (1)$$

$$LIVE_out(B) = \bigcup (S \in succ(B)) \; LIVE_in(S) \quad (2)$$

where $use(B)$ is the set of variables read before being written in B and $def(B)$ is the set written in B. Dead code elimination in Section VI-C is the practical use of this information.

E. Optimization Foundations

Constant propagation and folding originate in the global data-flow framework of Kildall [4], and global common subexpression elimination was introduced by Cocke [5]. Control flow analysis and loop identification are due to Allen [6]. Strength reduction replaces expensive operators by cheaper equivalents, and loop-invariant code motion hoists computations whose operands do not change inside a loop [1], [3]. Modern compilers organize such work around richer representations such as static single assignment form [7].

F. Educational Compiler Tools

Many teaching tools visualize only the front end, such as tokens and parse trees, or show a single fixed example. Others depend on reading the intermediate-code dumps of industrial compilers. This project instead concentrates on the middle and back end, and lets learners supply their own programs.

III. SYSTEM ARCHITECTURE

A. Technology Stack

The application is written in TypeScript on React 19 and bundled with Vite. Unit and integration tests run on Vitest, linting uses Oxlint, icons come from Lucide React, and all styling is hand-written vanilla CSS. No server is required: compilation, optimization, and verification all execute in the browser, so the tool works offline once loaded and never sends user code anywhere.

B. Pipeline Overview

The class `CompilerEngine` coordinates eight phases, shown in Fig. 1 and summarized in Table I. Each phase consumes the output of the previous one, and each module lives in its own directory under `src/compiler/`, which keeps the phases independently testable.

TABLE I
PIPELINE PHASES AND SOURCE MODULES

Phase	Module	Output
Lexical analysis	`Lexer.ts`	Token stream
Syntax analysis	`Parser.ts`	Abstract syntax tree
TAC generation	`TACGenerator.ts`	Linear TAC list
Basic blocks	`BasicBlock.ts`	Blocks B1, B2, ...
CFG	`CFG.ts`	Typed block edges
Expression DAG	`DAG.ts`	Per-block DAG
Optimization	`optimizations/`	Optimized TAC
Verification	`TACInterpreter.ts`	Match / mismatch

C. Design Principles

Three principles guided the design. First, *modularity*: every phase and every optimization is a separate module with a narrow interface, so a pass can be added, removed, or reordered without touching others. Second, *fidelity*: the algorithms follow the textbook formulations, so what learners see matches what they read. Third, *verifiability*: no transformation is trusted until the interpreter has confirmed that behavior is unchanged.

D. Intermediate Instruction Set

Table II lists the TAC instructions produced by the generator and consumed by the optimizer and interpreter.

TABLE II
TAC INSTRUCTION SET

Instruction	Form	Meaning
`ADD SUB MUL DIV MOD`	`x = y op z`	Arithmetic
`SHL SHR`	`x = y << k`	Bit shift
`ASSIGN`	`x = y`	Copy
`LABEL`	`Lk:`	Jump target
`GOTO`	`goto Lk`	Unconditional jump
`IF_FALSE_GOTO`	`ifFalse c goto Lk`	Conditional jump
`PRINT`	`print y`	Console output
`RETURN`	`return y`	End program

IV. FRONT END AND INTERMEDIATE CODE

A. Lexical Analysis

The lexer recognizes the `int` type, identifiers, integer and string literals, the arithmetic operators, the assignment and update operators (`=`, `++`, `--`, `+=`, `-=`), the six relational operators, the bitwise operators (`&`, `|`, `^`, `<<`, `>>`), the keywords `if`, `else`, `while`, `for`, `return`, `print`, and

`printf`, and both single-line and block comments. Each token records its line and column for later error reporting.

B. Syntax Analysis

A recursive-descent parser with precedence climbing builds the AST. The statement structure is summarized by the grammar in Fig. 2 (simplified).

```
stmt  ::= decl | assign | if | while
        | for | print | return | block
decl  ::= 'int' ID ['=' expr]
          {',' ID ['=' expr]} ';'
if    ::= 'if' '(' expr ')' stmt
          ['else' stmt]
while ::= 'while' '(' expr ')' stmt
expr  ::= unary {binop unary}
```

Fig. 2. Simplified grammar accepted by the parser.

Binary operators use C-style precedence, from multiplicative operators at the top, through additive, shift, relational, and equality operators, to the bitwise operators at the bottom. The parser supports comma-separated declarations such as `int a, b = 10;`, uninitialized variables, and compound assignments that are desugared during parsing. Syntax errors are reported with one-based line and column positions so that users can locate mistakes in their own code.

C. Three-Address Code Generation

The generator lowers the AST into TAC using the instructions of Table II. Before lowering, it scans the AST for every user variable, which guarantees that compiler temporaries (`t1`, `t2`, ...) can never collide with a name chosen by the user. Control constructs become labels and jumps. For example, the loop `while (i < n) { s = s + i; i = i + 1; }` is lowered, in illustrative notation, to:

```
L1:  ifFalse i < n goto L2
     t1 = s + i
     s  = t1
     t2 = i + 1
     i  = t2
     goto L1
L2:
```

The label `L1` marks the loop test and `L2` the exit. These labels and jumps are exactly the information the basic block builder uses next.

V. PROGRAM STRUCTURE ANALYSIS

A. Basic Block Construction

Blocks are formed by the standard leader rules, given in Algorithm 1: the first instruction is a leader, every jump target is a leader, and every instruction following a jump is a leader. Each block then extends from one leader up to, but not including, the next.

Algorithm 1. Leader identification.

```
leaders <- { first instruction }
for each instruction i in TAC:
  if i is GOTO or IF_FALSE_GOTO:
    leaders += target label of i
    leaders += instruction after i
split TAC at every leader -> B1..Bn
```

B. Control Flow Graph

The CFG records predecessors and successors for each block and classifies every edge, as listed in Table III. Fig. 3 shows the CFG of the loop above, with its initialization and return added. The back edge from B3 to B2 identifies a natural loop, which the loop-invariant code motion pass requires.

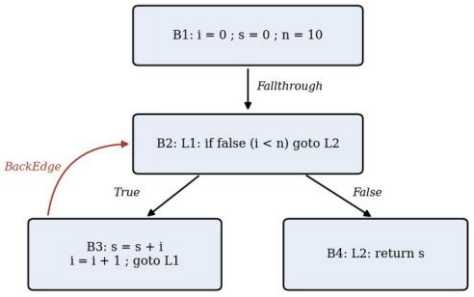


Fig. 3. CFG with typed edges for a program containing one while loop.

TABLE III
CFG EDGE CLASSES

Edge	When it is created
True	Branch taken when the tested condition holds
False	Branch taken at `IF_FALSE_GOTO` when the condition fails
Fallthrough	Control continues into the next block in order
BackEdge	Jump to a block that dominates its source, closing a loop

C. Expression DAG

For each basic block the system builds a DAG whose nodes are operands and operators and whose attached labels are the variables assigned to each node, following Algorithm 2. Commutative operators share nodes, so `a + b` and `b + a` map to the same node, as Fig. 4 illustrates.

Algorithm 2. Block DAG construction.

```
for each instruction x = y op z in B:
  ny <- node(y) ; nz <- node(z)
  if op is commutative: order (ny,nz)
  n <- existing node (op,ny,nz)
       or create a new one
  detach label x from its old node
  attach label x to n
```

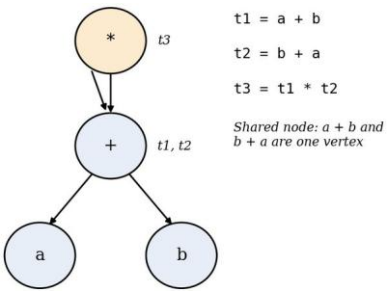


Fig. 4. DAG for a block with a commutative shared subexpression.

When constant folding reduces a whole block to a single constant return, the DAG view falls back to the pre-folded expression tree so that the learner can still see the original structure instead of a single leaf.

VI. OPTIMIZATION PASSES

Five passes, listed in Table IV, transform the TAC. Each pass is a separate module with a common interface, so users can enable any subset and step through them one at a time.

TABLE IV
OPTIMIZATION PASSES

Pass	Transformation
Constant folding and propagation	Evaluates constant expressions and forwards known values; guarded against division and modulo by zero
Common subexpression elimination	Reuses earlier results using canonical operand order; invalidates on redefinition; propagates intra-block copies
Dead code elimination	Backward liveness removes unread assignments; keeps `PRINT`, `RETURN`, and branches
Strength reduction	Powers of two become `SHL`/`SHR`; removes `*0`, `*1`, `/1`, `+0`, `-0`
Loop-invariant code motion	Fixed-point hoisting of loop-static computations into the preheader

A. Constant Folding and Propagation

The pass evaluates operations whose operands are literals at compile time and substitutes known constant values into later uses. Division or modulo by zero is never folded, so the original runtime behavior, including any run-time fault, is preserved. For example, `t1 = 10 * 2` followed by `t2 = t1 + 5` becomes `t1 = 20` and then `t2 = 25`.

Algorithm 3. Constant folding and propagation (per block).

```
env <- {}
for each instruction i in B:
  substitute known constants from env
  if i = x op c1 c2 and not
     (op in {DIV,MOD} and c2 = 0):
    i <- x = eval(op, c1, c2)
  if i = x = c: env[x] <- c
  else: remove x from env
```

B. Common Subexpression Elimination

Previously computed expressions are cached under a canonical operand order. A cache entry is invalidated as soon as one of its operands is redefined, which prevents stale reuse. Copy propagation within the block then removes the aliases that elimination creates. For example, the second computation in `t1 = a + b` and `t2 = b + a` is replaced by the copy `t2 = t1`.

Algorithm 4. Common subexpression elimination (per block).

```
table <- {}
for each instruction x = y op z:
  key <- canonical(op, y, z)
  if key in table: x = table[key]
  else: table[key] <- x
  invalidate entries that read x
  or are stored in x
```

C. Dead Code Elimination

A backward liveness scan, using the relations of Eqs. (1) and (2), marks which variables are later read. Assignments to variables that are never read are deleted, while instructions with side effects (output, returns, and branches) are always kept, because removing them would change observable behavior.

Algorithm 5. Dead code elimination (per block).

```
live <- LIVE_out(B)
for each instruction i, last to first:
  if i has a side effect:
    live += uses(i)
  else if def(i) in live:
    live -= def(i) ; live += uses(i)
  else: delete i
```

D. Strength Reduction

Multiplication or division by a power of two is rewritten as a left or right shift, so `x * 8` becomes `x << 3`. Algebraic identities are also simplified: `x * 1`, `x / 1`, `x + 0`, and `x - 0` become plain copies, and `x * 0` becomes the constant 0.

E. Loop-Invariant Code Motion

The pass follows the textbook formulation of Algorithm 6. It iterates to a fixed point because hoisting one instruction can make another invariant on the next iteration.

Algorithm 6. Loop-invariant code motion.

```
repeat
  changed <- false
  for each loop L (from back edges):
    for each instruction i in L:
      if operands of i are constant
         or defined outside L
         or already hoisted:
        move i to the preheader of L
        changed <- true
until not changed
```

For the loop of Section IV-C extended with a loop-static term, `t3 = k * 8` uses only `k`, which is not changed in the loop. After strength reduction it reads `t3 = k << 3`, and the pass moves it before label `L1` so it runs once instead of once per iteration.

F. Pass Interaction

The passes enable one another. Folding exposes dead assignments that elimination removes; strength reduction produces simple shifts that become candidates for hoisting; and common subexpression elimination creates copies that propagation then cleans up. Because the inspector runs one pass at a time, learners can see these effects, and also see that a different order can leave work undone.

VII. SEMANTIC VERIFICATION

Optimizations must never change what a program does. The `TACInterpreter` acts as a virtual machine that executes TAC directly, as summarized in Algorithm 7.

Algorithm 7. Semantic verification.

```
run P0 (unoptimized) up to 50,000 steps
run P1 (optimized)   up to 50,000 steps
if outputs equal and returns equal:
  report MATCH
else:
  report MISMATCH
```

The step cutoff guards against non-terminating user programs, such as an accidental infinite loop, so the browser never hangs. The verdict is displayed to the user, which turns every optimization run into a self-checking experiment. This is a testing-based argument rather than a proof: it establishes equivalence on the executed input, which is sufficient here because the language has no external input, so a program has exactly one behavior.

VIII. CORRECTNESS, COMPLEXITY, AND DESIGN DECISIONS

A. Soundness Conditions

Each pass is safe only if it preserves observable behavior, meaning printed output and the returned value. Table V states the condition each pass relies on.

TABLE V
SAFETY CONDITION OF EACH PASS

Pass	Condition that preserves behavior
Folding	Operands are known constants; no fold of division or modulo by zero
CSE	Operands unchanged between the two computations
DCE	Removed value is never read; side effects retained
Strength	Rewrite is equal for every integer input of the operand
LICM	Moved value does not depend on loop-modified variables

B. Complexity

Table VI gives the typical asymptotic cost of the textbook algorithms used, where n is the number of TAC instructions and L the number of loops. All phases are low-order polynomial, which keeps interaction instantaneous for the program sizes used in teaching.

TABLE VI
TYPICAL COST OF EACH PHASE

Phase	Typical cost
Lexing, parsing, TAC	Linear in source size
Basic blocks and CFG	Linear in n
Block DAG	Linear in block size with hashing
Folding, CSE, DCE	Linear per pass over n
LICM	$O(L \cdot n)$ per fixed-point round
Interpreter	At most 50,000 steps

C. Design Decisions

Several decisions address problems that appear when users write arbitrary programs:

- **Name collisions.** User variables are collected before lowering so compiler temporaries never shadow them.
- **Run-time faults.** Folding skips division and modulo by zero, leaving the fault to occur at run time as in the original.
- **Non-termination.** The interpreter's 50,000-step cutoff keeps the browser responsive on infinite loops.
- **Stale facts.** The CSE cache is invalidated on redefinition, avoiding incorrect reuse.
- **Readable graphs.** Commutative node sharing and the pre-folded DAG fallback keep diagrams informative.

IX. VISUALIZATION AND USER INTERFACE

The interface is a dark-mode editor with a line-number gutter and quick-insert buttons for common syntax. Pass selection toggles let users choose which optimizations run. The step-by-step inspector provides *Reset to Unoptimized*, *Previous Pass*, *Next Pass*, and *Finish All* controls. Three components present the compiler's internal state, as listed in Table VII.

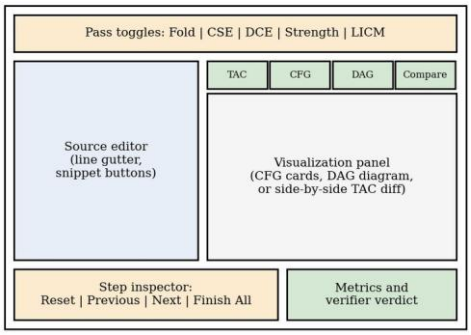


Fig. 5. Schematic layout of the Playground interface.

TABLE VII
VISUALIZATION COMPONENTS

Component	Purpose
`CFGVisualizer`	Basic block cards and the typed transitions between them
`DAGVisualizer`	SVG node-link diagram with depth leveling and highlighted shared subexpressions
`ComparisonView`	Line-by-line diff of unoptimized and optimized TAC with instruction-reduction metrics

A typical session proceeds as follows: the user edits or selects a program, the engine compiles it, the user inspects the TAC, CFG, and DAG, steps through the passes while watching the diff, and finally reads the verifier verdict. A built-in sample library, including the faculty benchmark as the default preset, gives beginners a starting point.

X. IMPLEMENTATION AND TESTING

A. Repository Organization

Compiler logic is separated from presentation. The directory `src/compiler/` contains the lexer, parser, TAC generator, basic block and CFG builders, DAG builder, optimization passes, and interpreter, with no dependence on React. The directory `src/components/` contains the visualizers, `src/samples/` holds example programs, and `src/tests/` holds the test suites. This separation allows the compiler to be tested without a browser.

B. Core Data Structures

Conceptually, the compiler passes a small number of plain data structures between phases:

- **Token:** kind, text, line, and column.
- **AST node:** node type, child nodes, and source position.
- **TAC instruction:** opcode, up to two operands, and a result or target label.
- **Basic block:** identifier, instruction list, predecessors, and successors.
- **CFG edge:** source, target, and class (True, False, Fallthrough, BackEdge).
- **DAG node:** operator or operand, child nodes, and attached variable labels.

Keeping these structures simple lets each visualizer render directly from the data a phase produced, with no hidden state between the compiler and the interface.

C. Test Strategy

The Vitest suites exercise the compiler core and custom user-input scenarios, ensuring that programs typed by users, and not only built-in samples, are handled correctly. Because

the verifier already compares optimized and unoptimized behavior, test programs double as end-to-end equivalence checks. Three quality gates, summarized in Table VIII, must pass together.

TABLE VIII
VERIFICATION SUMMARY

Check	Result
Vitest (`npm test`)	34 of 34 tests passing, 4 suites
Oxlint (`npm run lint`)	0 errors, 0 warnings
Build (`npm run build`)	Clean production build
Semantic verifier	Outputs and return values match on benchmark

XI. RESULTS AND DISCUSSION

A. Faculty Benchmark

The default preset is the faculty benchmark program:

```
int a = 10;
int b = 20;
int x = 10 * 2 + 5;
int y = x + x;
return y;
```

Constant folding evaluates `10 * 2 + 5` to 25 and propagates it, so `y = x + x` folds to 50. Dead code elimination then removes the now-unread assignments to `a`, `b`, `x`, and `y`. The optimizer reduces the original eight TAC instructions to one, `return 50`, and the verifier confirms that both versions return 50. Using the instruction-reduction metric

$$R = (N_0 - N_1) / N_0 \times 100\% \quad (3)$$

with $N_0 = 8$ and $N_1 = 1$, the reduction is $R = 87.5\%$.

B. Loop Example

To show hoisting, consider a program that adds `k * 8` on each of ten iterations. In illustrative notation, the unoptimized TAC and the result after strength reduction and loop-invariant code motion are shown below. The exact sequence in the tool depends on the enabled passes.

```
Before:
  L1: ifFalse i < 10 goto L2
      t1 = k * 8
      t2 = s + t1
      s  = t2
      t3 = i + 1
      i  = t3
      goto L1
  L2: return s


After:
      t1 = k << 3       (hoisted)
  L1: ifFalse i < 10 goto L2
      t2 = s + t1
      s  = t2
      t3 = i + 1
      i  = t3
      goto L1
  L2: return s
```

The loop body shrinks from five assignments to four, and the shift now executes once instead of ten times. The CFG view shows the new instruction in the block before the loop header, and the verifier confirms that the returned sum is unchanged.

C. Static and Dynamic Metrics

Two measures describe an optimization. The static count N is the number of TAC instructions in the program text and drives Eq. (3). The dynamic count D is the number of instructions the interpreter executes, which accounts for loop repetition. The dynamic reduction is defined analogously:

$$R_D = (D_0 - D_1) / D_0 \times 100\% \quad (4)$$

In the loop example the static count is unchanged, because the hoisted instruction is moved rather than removed. Ignoring the identical initialization, a run of ten iterations executes 72 instructions before the transformation and 63 after, giving $R_D = 12.5\%$. This is an illustrative calculation, but it shows why the dynamic view matters.

D. Discussion

The benchmark shows how passes cooperate: folding creates dead code that elimination removes. The loop example shows a different benefit, since hoisting does not shrink the program text but lowers the work done per iteration, a distinction that static instruction counts alone cannot show. The tool therefore reports both the instruction reduction and the verified behavior.

From a teaching standpoint, the strengths are immediacy and honesty: users see real results on their own code, and a failed verification would expose an unsound pass rather than hide it. A limitation of the evaluation is that it relies on the benchmark and automated tests; a classroom study measuring learning outcomes has not yet been carried out.

E. Threats to Validity

- The quantitative evidence rests on one benchmark and a set of small programs, so the reduction figures should not be generalized.
- Instruction counts are proxies for speed; the interpreter counts abstract steps, not machine time.
- The language is small, so conclusions about it may not transfer to full languages.
- Verification by execution establishes equivalence only for the program behavior actually run.

XII. EXTENDED CASE STUDIES

This section walks through further small programs, one per optimization, in illustrative TAC notation. They match the experiments a learner can reproduce with the toggles and the step inspector.

A. Common Subexpression Elimination

Two operand orders of the same sum are computed and then multiplied:

```
Before:
  t1 = a + b
  t2 = b + a
  t3 = t1 * t2
After CSE and copy propagation:
  t1 = a + b
  t3 = t1 * t1
```

Canonical operand ordering recognizes `b + a` as the same expression as `a + b`. Elimination first rewrites `t2` as a copy of `t1`; copy propagation then substitutes it into the product, and dead code elimination removes the unused copy. The DAG of Fig. 4 shows the same fact graphically.

B. Dead Code and Side Effects

The next program mixes useful and useless assignments:

```
Before:
  x = 5
  y = x * 2
  print y
  z = y + 1
  return 0
After folding and DCE:
  print 10
  return 0
```

Folding turns `y` into 10 and propagates it into `print`. The assignments to `x`, `y`, and `z` are then never read and are removed, but `print` and `return` are kept because they are observable. This case shows why side-effecting instructions must be treated as roots of liveness.

C. Algebraic Simplification and Strength Reduction

```
Before:
  t1 = n * 8
  t2 = n * 1
  t3 = t2 + 0
  t4 = t1 / 4
After:
  t1 = n << 3
  t2 = n
  t3 = t2
  t4 = t1 >> 2
```

Identity operations become copies, which later passes can remove. The rewrite of division by a power of two deserves care: for negative operands an arithmetic right shift rounds toward negative infinity, whereas integer division in C-like languages truncates toward zero. Production compilers add a correction for signed values. In this tool the verifier provides the safety net, since any divergence in printed output or return value is reported as a mismatch.

D. Folding Without Changing Faults

An expression such as `10 / 0` has no compile-time value. The folding pass leaves it in place, so the program fails or is cut off at run time exactly as before. Folding it to a constant would turn a faulty program into a well-behaved one, which is a change of meaning rather than an optimization.

E. Sensitivity to Pass Order

Running dead code elimination before folding removes little, because the assignments still look live; running it after folding removes nearly everything in the benchmark. Conversely, strength reduction before loop-invariant code motion lets the hoisted instruction be the cheaper shift. These orderings are easy to try with the inspector and illustrate why optimizers repeat passes or order them carefully [2], [3].

F. End-to-End Trace

Finally, a complete trace of a tiny program through the pipeline shows how the phases connect:

```
Source:
  int a = 2; int b = a * 4; print b;
Tokens:
  int a = 2 ; int b = a * 4 ;
  print b ;
TAC:
  a = 2
  t1 = a * 4
  b = t1
  print b
Blocks and CFG:
  one block B1, no edges (no jumps)
DAG of B1:
  leaves 2 (a) and 4; node * (t1, b)
After folding and DCE:
  print 8
```

Because the program has no jumps, it forms a single block and the CFG is a single node. Folding replaces `a * 4` by 8, and everything except the observable `print` becomes dead. The verifier then confirms that both versions print 8.

XIII. DIAGNOSTICS AND ERROR HANDLING

A. Compile-Time Diagnostics

Lexical and syntax errors carry the one-based line and column of the offending token. An illustrative message has the form:

```
Syntax error at line 3, column 14:
  expected ';' after expression
```

Because the editor shows a line-number gutter, learners can jump straight to the problem. Accurate positions matter in a teaching tool, since beginners are more likely to write invalid programs than experienced users.

B. Verification Outcomes

The verification workflow is shown in Fig. 6. A *match* means the optimized program printed the same values and returned the same value as the baseline within the step limit. A *mismatch* signals a defect in a pass, and it is shown rather than hidden, so that bugs introduced while extending the optimizer are caught at once. If a program reaches the 50,000-step cutoff, execution stops at the cutoff instead of freezing the page.

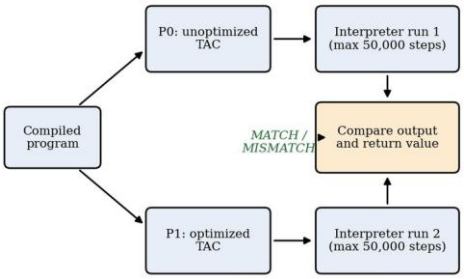


Fig. 6. Semantic verification workflow.

XIV. COMPARISON WITH EXISTING APPROACHES

Table IX contrasts the Playground with two broad families of resources that students commonly meet: the intermediate-code dumps of production compilers and visualizers that cover only the front end. The entries describe typical behavior of those families, not a survey of specific products.

TABLE IX
COMPARISON WITH TYPICAL ALTERNATIVES

Criterion	Compiler IR dumps	Front-end visualizers	Playground
Runs in a browser	Rarely	Often	Yes
User-written code	Yes	Often	Yes
Block-level CFG	Extra tools	Rare	Built in
Expression DAG	Rare	Rare	Built in

Criterion	Compiler IR dumps	Front-end visualizers	Playground
Step through passes	Via flags	No	Yes
Behavior check	External tests	No	Built in

The principal advantage is the combination: one environment in which a learner writes code, watches each transformation, and receives an automatic correctness verdict. The principal trade-off is scope. The Playground covers a small language and five classical passes, whereas production compilers support full languages and hundreds of transformations.

XV. DEPLOYMENT AND REPRODUCIBILITY

The project is a standard Node-based web application. Table X lists the commands used to run, test, and build it. Since compilation happens in the browser, the production build is a static site that can be served by any static host.

TABLE X
PROJECT COMMANDS

Command	Purpose
`npm run dev`	Development server on `localhost:5173`
`npm test`	Run the Vitest suites
`npm run lint`	Run Oxlint
`npm run build`	Create the production build

Privacy. User programs are processed locally and are not transmitted to a server, so students can experiment freely. *Reproducibility.* The test suites and the default benchmark give a fixed reference: after any change, the same program must still reduce to `return 50` with a matching verdict.

XVI. EDUCATIONAL USE

The tool supports guided laboratory exercises. Table XI lists experiments that map directly to the controls described above.

TABLE XI
SUGGESTED LABORATORY EXPERIMENTS

Experiment	What the learner observes
Run the benchmark with one pass at a time	Which pass contributes which reduction
Disable DCE after folding	Dead assignments remain in the output
Write a loop with a loop-static product	The product moves before the loop
Write `a + b` and `b + a`	One shared node in the DAG
Introduce a syntax error	Line and column of the error
Write an infinite loop	Verifier stops at its step cutoff

Because every graph and listing is generated from the learner's own program, the same exercises can be repeated with new inputs, which supports exploration beyond a fixed set of examples.

A. A Possible Lesson Plan

A single laboratory session can follow three steps. In the first, students run the faculty benchmark and predict, before pressing the button, what each pass will do; the inspector then confirms or corrects the prediction. In the second, they write a program of their own that exercises one pass, such as a loop with an invariant product, and explain the CFG and DAG that appear. In the third, they deliberately break an assumption, for example by dividing by zero or looping forever, and describe how the tool responds.

B. Assessment Ideas

Instructors can ask students to hand in a program together with a short explanation of the optimized output, to predict the instruction count before running the optimizer, or to identify which pass produced a given change in the diff view. Since the verifier gives immediate feedback, students can check their own reasoning without waiting for a grader.

XVII. LESSONS LEARNED

Several lessons emerged from building the tool. First, *separating the compiler from the interface* made the compiler testable without a browser and allowed the visualizers to be simple consumers of data.

Second, *an executable reference changes how optimizers are developed*. With the interpreter available, a pass can be modified and its effect on behavior checked immediately, which encourages experimentation and exposes mistakes early.

Third, *users write unexpected programs*. Uninitialized variables, division by zero, and non-terminating loops are all natural for beginners to produce, and each of them drove a concrete design decision described in Section VIII. A teaching compiler must therefore handle bad input gracefully rather than assume well-formed programs.

Fourth, *showing intermediate results matters more than showing final ones*. The benchmark ends in a single instruction, but the learning value lies in the sequence of states that leads there, which is why the step inspector and per-pass diff were central to the design.

XVIII. LIMITATIONS AND FUTURE WORK

The source language is intentionally small: it supports integer variables, arithmetic, relational and bitwise operators, conditionals, and loops, but not arrays, functions, or pointers. The optimizer works at the level of basic blocks and simple natural loops, and the verifier checks behavior by execution rather than by formal proof. Planned extensions are:

- logical operators and the ternary operator;
- arrays and functions with inter-procedural analysis;
- additional passes such as copy propagation, global value numbering, and loop unrolling;
- static single assignment form as an additional representation;
- highlighting of the instructions changed by each pass; and
- export of the CFG and DAG as images, and a classroom evaluation.

Language. Adding logical operators with short-circuit evaluation would require new control-flow lowering and would make the CFGs more interesting. Arrays and functions would introduce memory and call semantics, and with them the need for alias and inter-procedural analysis.

Analysis. Global value numbering and static single assignment form [7] would let optimizations work across

blocks, and a dominator-tree view would make the definition of a back edge explicit for students.

Interface. Highlighting the instructions each pass changed, animating the transition between steps, and exporting graphs would improve usability. A shareable link that encodes a program would let instructors distribute exercises.

Evaluation. A controlled classroom study comparing learning outcomes with and without the tool would provide the evidence that the present work lacks.

XIX. CONCLUSION

The Compiler Optimization Playground demonstrates a complete compiler back end that is both genuine and transparent. By accepting user-written programs, applying five real optimization passes, and verifying correctness by execution, it lets students observe compiler theory in action. Its modular TypeScript design, 34 passing tests, and clean lint and build status make it a dependable base for further extensions.

APPENDIX: ABBREVIATIONS

TABLE XII
ABBREVIATIONS USED IN THIS PAPER

Term	Meaning
AST	Abstract syntax tree
CFG	Control flow graph
CSE	Common subexpression elimination
DAG	Directed acyclic graph
DCE	Dead code elimination
LICM	Loop-invariant code motion
TAC	Three-address code
VM	Virtual machine (TAC interpreter)

ACKNOWLEDGMENT

The authors thank the faculty of the Compiler Design Laboratory (BCSE307P) for guidance and review.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd ed. Burlington, MA, USA: Morgan Kaufmann, 2011.

[3] S. S. Muchnick, *Advanced Compiler Design and Implementation*. San Francisco, CA, USA: Morgan Kaufmann, 1997.

[4] G. A. Kildall, "A unified approach to global program optimization," in *Proc. 1st ACM Symp. Principles of Programming Languages (POPL)*, Boston, MA, USA, 1973, pp. 194–206.

[5] J. Cocke, "Global common subexpression elimination," *ACM SIGPLAN Notices*, vol. 5, no. 7, pp. 20–24, 1970.

[6] F. E. Allen, "Control flow analysis," *ACM SIGPLAN Notices*, vol. 5, no. 7, pp. 1–19, 1970.

[7] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently computing static single assignment form and the control dependence graph," *ACM Trans. Program. Lang. Syst.*, vol. 13, no. 4, pp. 451–490, 1991.